Mahidol University
Wisdom of the Land

Mahidol University

ITCS212 Web Programming

Node.js Part II

Instructor: Wudhichart Sawangphol
Email: wudhichart.saw@mahidol.edu
Instructor: Jidapa Kraisangka
Email: jidapa.kra@mahidol.edu

Course Objective

- Able to do routing with Node.js
- Able to do form handling with Node.js
- Able to create cookies and sessions using Node.js
- Able to connect Node.js with the relational database

Outline

- Express Framework (RECAP)
- express.Router()
- Form Handling (`GET` and `POST`)
- Cookies + Session
- Environmental Variables
- Database + Node.js

Express Framework

- **Express** is a minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications.
- Help handling things like routing
- Help building API

Routing with Express

- Initialize the project by

```
npm init
npm install express --save
npm install nodemon --save (Optional)
```

- (Optional) Set up the `script` in `package.json`, i.e.,

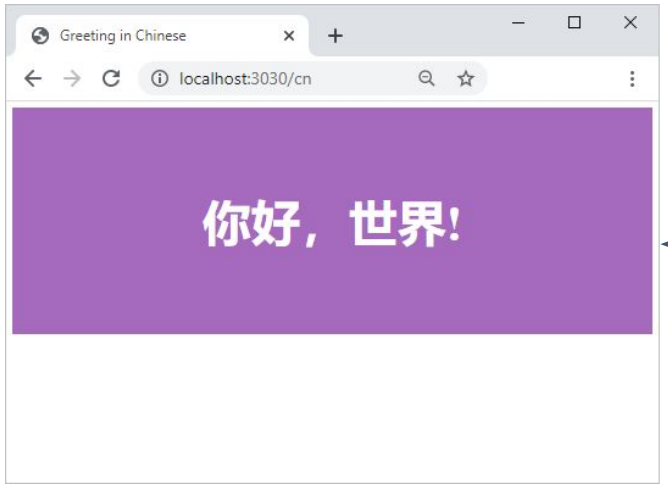
```
"start": "nodemon app.js"
```

- Create some HTML files for serving the request
- Create the main file, e.g. `app.js`, and do routing

Example: Routing

http://localhost:3030/cn

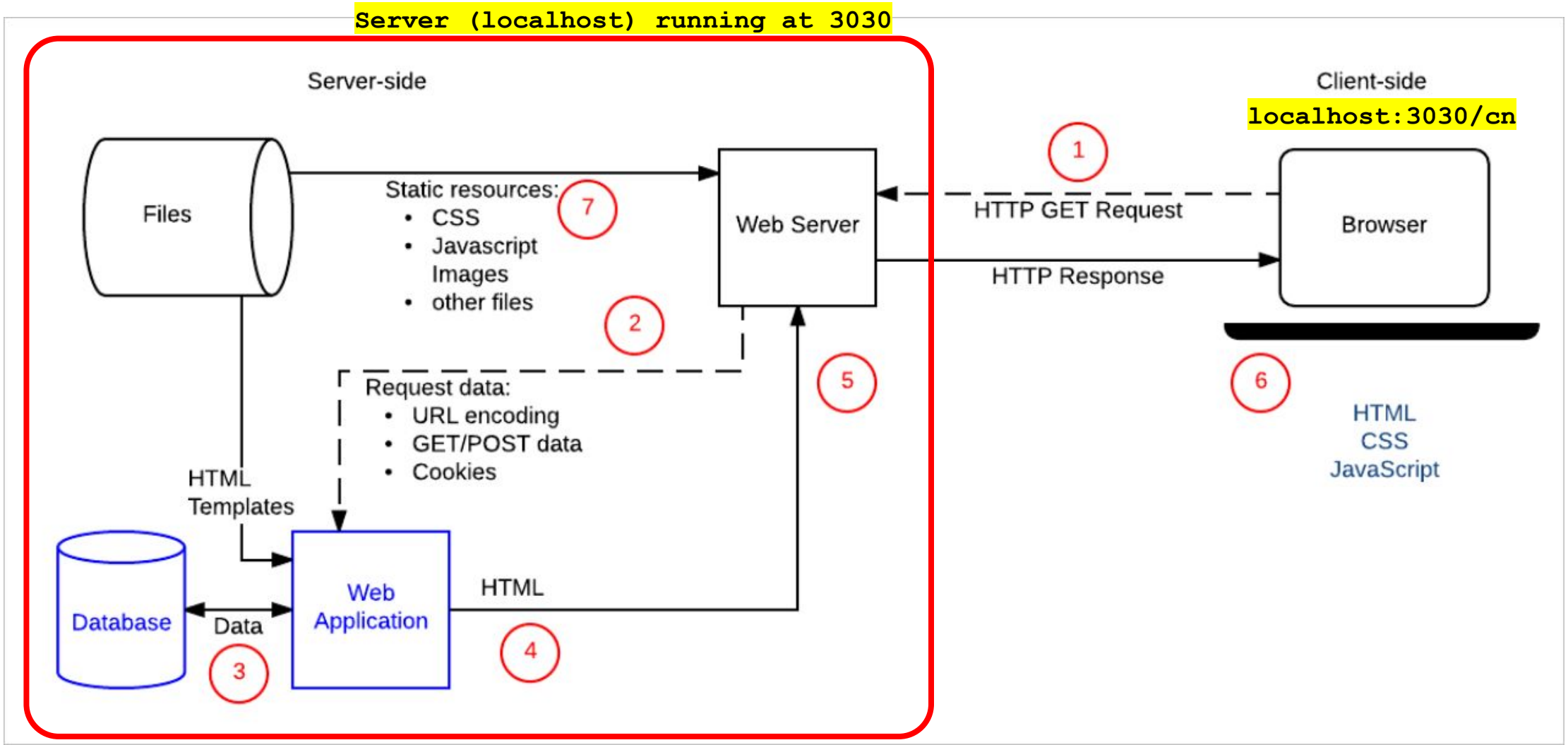
GET request at /cn




```
const path = require('path');
const express = require('express');
const app = express();

app.get('/',function(req,res){
  res.status(200).send("Hello World! In plain text");
});

app.get('/th',function(req,res){
  res.sendFile(path.join(__dirname+'/greeting_th.html'));
});

app.get('/cn', function(req, res){
  res.sendFile(path.join( __dirname+'/greeting_cn.html'));
});

app.use((req, res, next) => {
  console.log("404: Invalid accessed");
  res.status(404).send("Where are you going?");
})

app.listen(3030);
console.log('Running at Port 3030');
```

Global object

Web Page Request / Response

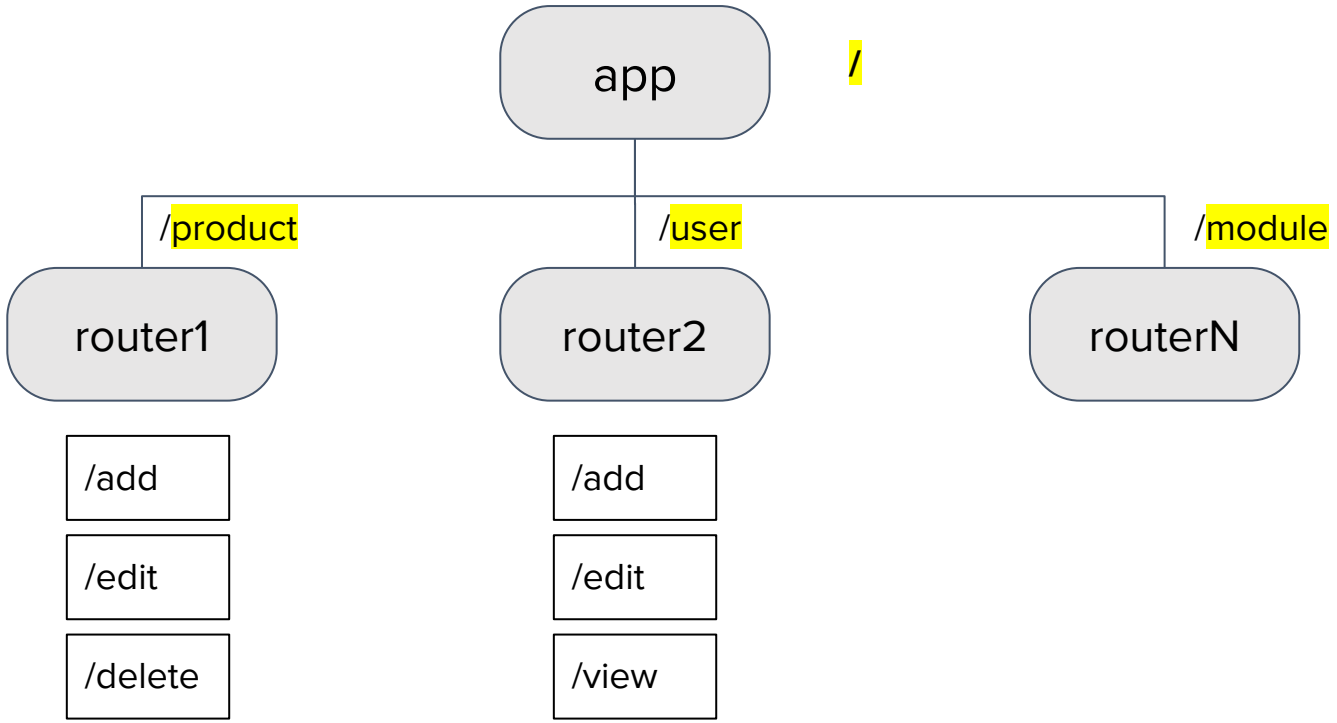


Source: https://developer.mozilla.org/en-US/docs/Learn/Server-side/First_steps/Introduction

express.router Middleware

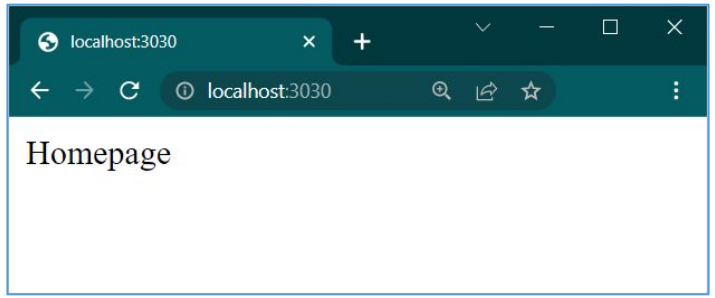
When the server become complicate, i.e. many routes, using `express.router()` middleware can help to manage the group of routes.

Middleware: Functions that have access to the request object (req) and the response object (res)



Example: Routing

```
const express=require('express');
const app=express();
const router1=express.Router();
const router2=express.Router();
app.use("/product", router1);      //Register router1
app.use("/user", router2);   //Register router2

// Main app
app.get('/', function (req, res){
    res.send("Homepage");
});
// Module: product
router1.get('/', function (req, res) {
    res.send("Module for product - Root");
});
router1.get('/add', function (req, res) {
    res.send("Module for product - Add");
});

// Module: user
router2.get('/', function (req, res) {
    res.send("Module for user - Root");
});
router2.get('/add', function (req, res) {
    res.send("Module for user - Add");
});

// Server running on the port: 3030
app.listen(3030, function () {
    console.log("Server listening at Port 3030");
});
```



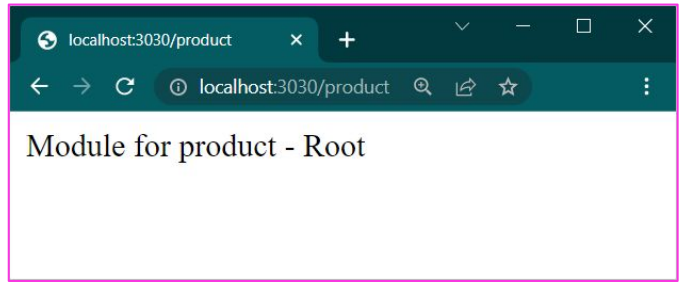
http://localhost:3030/

- http://localhost:3030/**product**
  - http://localhost:3030/**product**/add
- http://localhost:3030/**user**
  - http://localhost:3030/**user**/add

Example: Routing


```
const express=require('express');
const app=express();
const router1=express.Router();
const router2=express.Router();
app.use("/product", router1);      //Register router1
app.use("/user", router2);   //Register router2

// Main app
app.get('/', function (req, res){
    res.send("Homepage");
});
// Module: product
router1.get('/', function (req, res) {
    res.send("Module for product - Root");
});
router1.get('/add', function (req, res) {
    res.send("Module for product - Add");
});

// Module: user
router2.get('/', function (req, res) {
    res.send("Module for user - Root");
});
router2.get('/add', function (req, res) {
    res.send("Module for user - Add");
});

// Server running on the port: 3030
app.listen(3030, function () {
    console.log("Server listening at Port 3030");
});
```

http://localhost:3030/

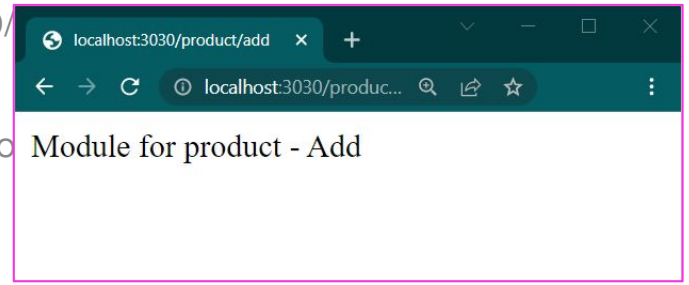


http://localhost:3030/**product**

http://localhost:3030/**product**/add

http://localhost:3030/

http://localho

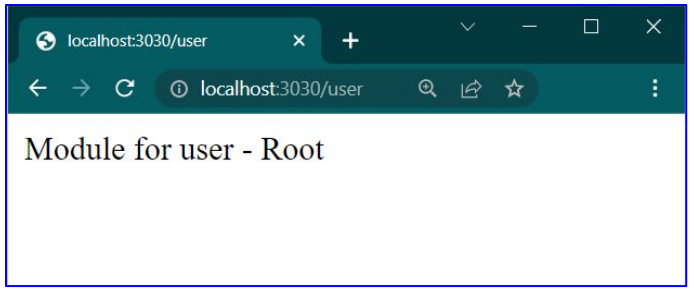


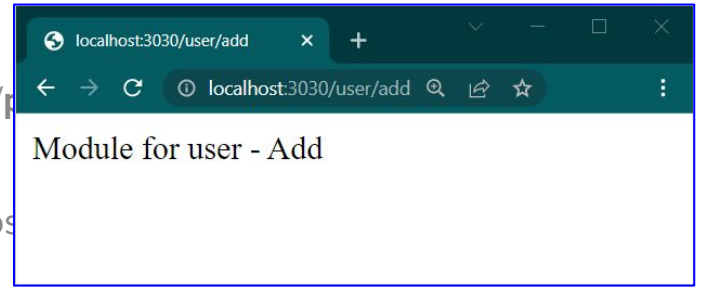
Example: Routing

```
const express=require('express');
const app=express();
const router1=express.Router();
const router2=express.Router();
app.use("/product", router1);      //Register router1
app.use("/user", router2);   //Register router2

// Main app
app.get('/', function (req, res){
    res.send("Homepage");
});
// Module: product
router1.get('/', function (req, res) {
    res.send("Module for product - Root");
});
router1.get('/add', function (req, res) {
    res.send("Module for product - Add");
});

// Module: user
router2.get('/', function (req, res) {
    res.send("Module for user - Root");
});
router2.get('/add', function (req, res) {
    res.send("Module for user - Add");
});

// Server running on the port: 3030
app.listen(3030, function () {
    console.log("Server listening at Port 3030");
});
```

http://localhost:3030/

- http://localhost:3030/p
  - http://localhos
- http://localhost:3030/**user**
  - http://localhost:3030/**user**/add





Mahidol University

Wisdom of the Land

Mahidol University

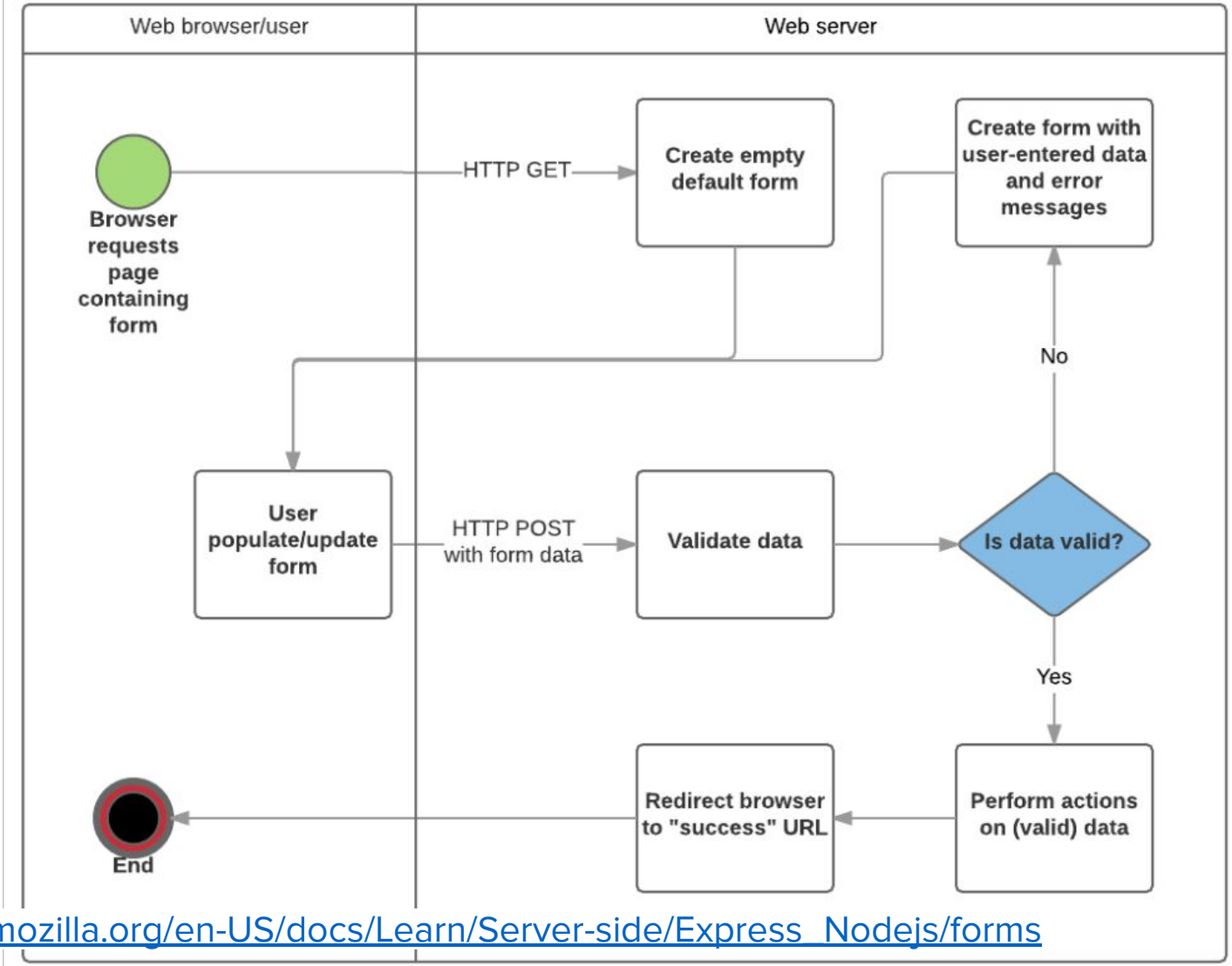
Form Handling

Form Handling

- **Form** is the place that users can enter the information.
- User input should be validated on the browser (client) whenever possible (i.e., JS script, HTML5 attributes)
- The form is submitted to the server by `GET` or `POST`
- After submission, **server handle values from a form**, e.g., add a new user for registration, update the existing product
- Extra validation at the server is considered if you intends to manage data to the database.

Ref: https://developer.mozilla.org/en-US/docs/Learn/Server-side/Express_Nodejs/forms

More on Form Handling



Ref: https://developer.mozilla.org/en-US/docs/Learn/Server-side/Express_Nodejs/forms

Example: Form Handling

Main step for the form handling in Node.js using Express

1. Prepare the Express framework

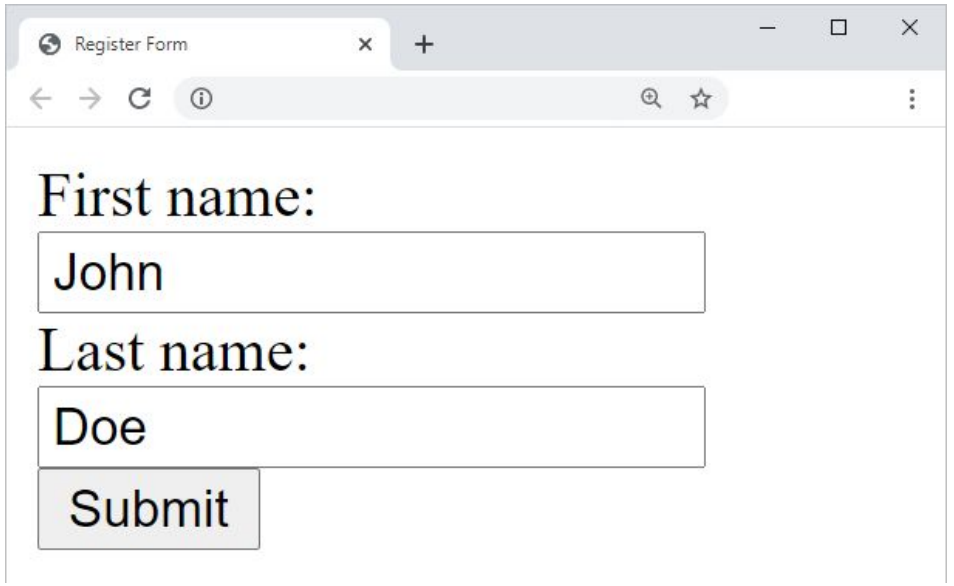
```
npm init
npm install express --save
npm install nodemon --save
```
(Optional)

2. Prepare the HTML form file (i.e., this form will be returned when requested)
   - Set the destination path in the attribute action
   - Set the method for submission (`GET` or `POST`)
3. Create the server (i.e., run on the port) in **`app.js`**
   - Handle some routing
   - Processing the form values after the form submission based on the HTTP method defined in (2)

Step 2: Preparing a HTML Form

myForm.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Register Form</title>
</head>
<body>
  <form id="frmRegister"  action="" method="">
    <label for="fname">First name:</label><br>
    <input type="text" name="fname"
    value="John"><br>
    <label for="lname">Last name:</label><br>
    <input type="text" name="lname"
    value="Doe"><br>
    <button type="submit" >Submit</button>
  </form>
</body>
</html>
```

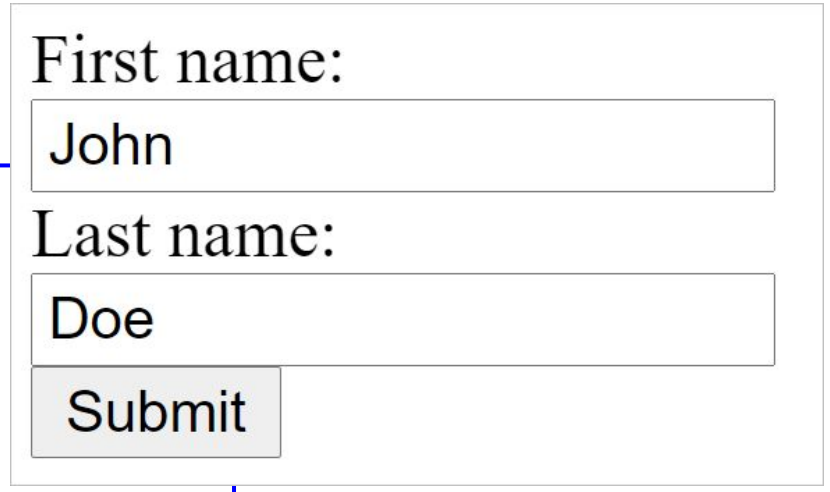


action the destination of the form submission
method the HTTP methods for the form submission

Step 2: HTML Form with **GET**

myForm.html

```
<!DOCTYPE html>
<html lang="en">
    <head>
        <title>Register Form</title>
    </head>
    <body>
      <form id="frmRegister" action="/form-get" method="GET">
            <label for="fname">First name:</label><br>
            <input type="text" name="fname" value="John"><br>
            <label for="lname">Last name:</label><br>
            <input type="text" name="lname" value="Doe"><br>
            <button type="submit">Submit</button>
      </form>
    </body>
</html>
```




```
(root)
> node_modules
|_package-lock.json
|_package.json
| app.js
|_myForm.html
```

Step 3: Create the server

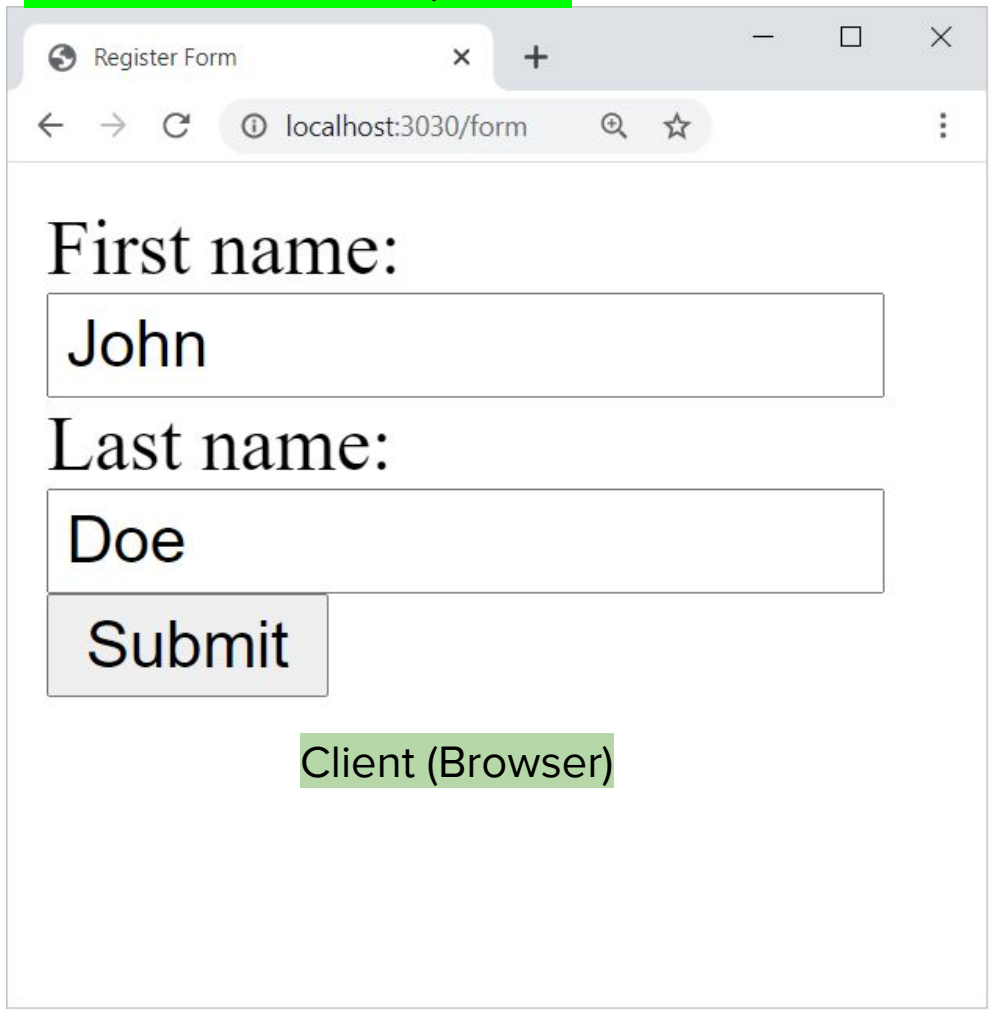
app.js

```
const path = require("path");
const express = require("express");
const app = express();

/* Use Router object to handle routes */
const router = express.Router();
app.use("/", router); // Register the router

// Handle GET: Display myform.html
router.get("/form",function(req,res){
    console.log("Send a form");
    res.sendFile(path.join(__dirname+'/myForm.html'));
});

// Server running on the port: 3030
app.listen(3030, function () {
    console.log("Server listening at Port 3030");
});
```

```
C:\[PATH]\form_handling> npm start

> form_handling@1.0.0 start C:
[PATH]\form_handling
> nodemon app.js

[nodemon] 2.0.15
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node app.js`
Server listening at Port 3030
_
```

Step 3: Create the server (cont.)

localhost:3030/form



Client (Browser)

```
C:\[PATH]\form_handling> npm start

> form_handling@1.0.0 start C: [PATH]\form_handling
> nodemon app.js

[nodemon] 2.0.15
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node app.js`
Server listening at Port 3030
Send a form
_
```

Server (Terminal)

Step 3: Handling GET request

app.js (Handling the form submission with GET)

```
const path = require("path");
const express = require("express");
const app = express();

/* Use Router object to handle routes */
const router = express.Router();
app.use("/", router); // Register the router

/* Handle GET: Display myform.html */
router.get("/form",function(req,res){
    console.log("Send a form");
    res.sendFile(path.join(__dirname+'/myForm.html'));
});

/* Handle GET: Form submitted by GET */

/* Server running on the port: 3030
app.listen(3030, function () {
    console.log("Server listening at Port 3030");
});
```

```
/* Handle GET: Form submitted by GET */
router.get("/form-get",function(req,res){
    console.log(`${req.method}`);
    console.log(req.query);  // JSON
    const fname = req.query.fname;
    const lname = req.query.lname;
    res.send(`Form submitted by
    ${fname} ${lname} with ${req.method}`);
});
```

The path (i.e., **/form-get**) must match the one defined in `<form> action` attribute.

req.query contains the URL query parameters (after the ? in the URL).

fname and **lname** are from `"name"` attribute in `<input>`

Step 3: Handling GET request

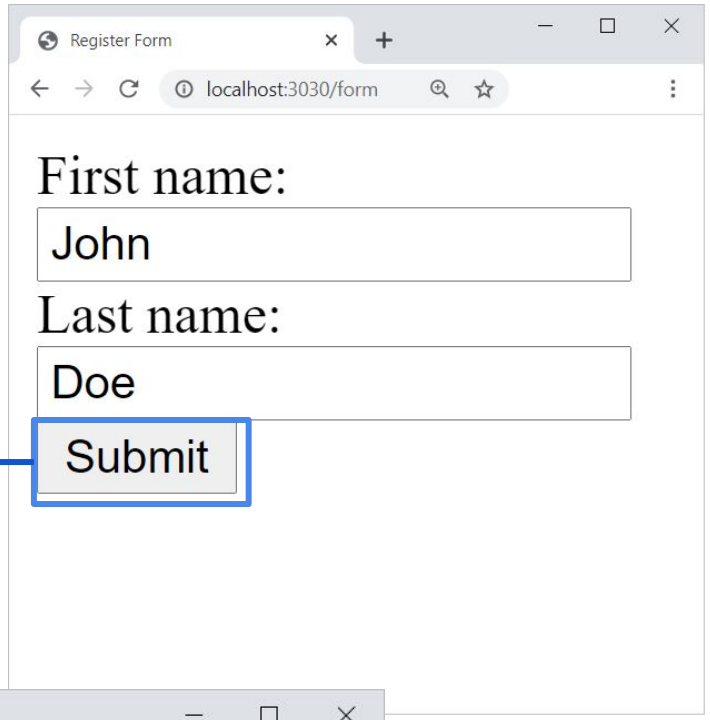

```
[nodemon] 2.0.15
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node app.js`
Server listening at Port 3030
Send a form
GET
{ fname: 'John' lname: 'Doe' },
```

HTTP GET
localhost:3030/form

Routing to myForm.html

HTTP GET
localhost:3030/form-get
?fname=John&lname=Doe

Form Submission

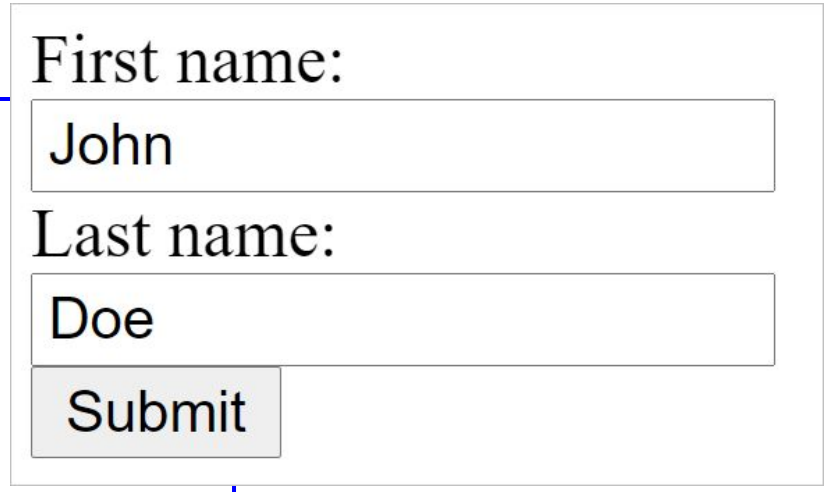


Show Response

Step 2: HTML Form with **POST**

myForm.html

```
<!DOCTYPE html>
<html lang="en">
    <head>
        <title>Register Form</title>
    </head>
    <body>
      <form id="frmRegister" action="/form-post"
       method="POST">
            <label for="fname">First name:</label><br>
            <input type="text" name="fname" value="John"><br>
            <label for="lname">Last name:</label><br>
            <input type="text" name="lname" value="Doe"><br>
            <button type="submit">Submit</button>
      </form>
    </body>
</html>
```




```
(root)
> node_modules
|_package-lock.json
|_package.json
| app.js
|_myForm.html
```

Step 3: Handling **POST** request

- Unlike **HTTP GET** request, **HTTP POST** requests encode the data in the message body

- *Earlier* version of Express requires an additional **middleware** module, called "`body-parser`", to extract incoming data of a **POST** request.

```
npm install body-parser --save
```

- Import "**body-parser**" and let Express router use the middleware

```
const bp = require("body-parser");
...
router.use(bp.json());
router.use(bp.urlencoded({ extended: true }));
```

package.json

```
...
"dependencies": {
  "body-parser": "^1.19.0",
  "express": "^4.17.1",
  "nodemon": "^2.0.7"
}
```

Ref: https://expressjs.com/en/guide/using-middleware.html

Step 3: Handling POST request

- After Express version 4.16, the "`body-parser`" was adopted in **`express`**, so the code can be simplified, as followed.

```
router.use(express.json());
router.use(express.urlencoded({ extended: true }));
```

package.json

```
...
"dependencies": {
  "body-parser": "^1.19.0",
  "express": "^4.17.1",
  "nodemon": "^2.0.7"
}
```

Note

- The **`express.json()`** is a built-in middleware function in Express. It parses incoming requests with JSON payloads and is based on **`body-parser`**
- The **`express.urlencoded()`** function is a built-in middleware function in Express. It parses incoming requests with urlencoded payloads and is based on **`body-parser`**.

Ref: https://www.geeksforgeeks.org/express-js-express-json-function/
https://www.geeksforgeeks.org/express-js-express-urlencoded-function/

Step 3: Handling POST request

app.js (Handling the form submission with POST)

```
const path = require("path");
const express = require("express");
const app = express();

/* Use Router object to handle routes */
const router = express.Router();
app.use("/", router); // Register the router

router.use(express.json());
router.use(express.urlencoded({ extended: true }));

/* Handle GET: Display myform.html */
router.get("/form",function(req,res){...});

/* Handle POST: Form submitted by POST */

/* Server running on the port: 3030
app.listen(3030, function () {...});
```

```
/* Handle POST: Form submitted by POST */
router.post("/form-post",function(req,res){
    console.log(req.method);
    console.log(req.body);
    const fname = req.body.fname;
    const lname = req.body.lname;
    res.send(`Form submitted by
    ${fname} ${lname} with ${req.method}`);
});
```

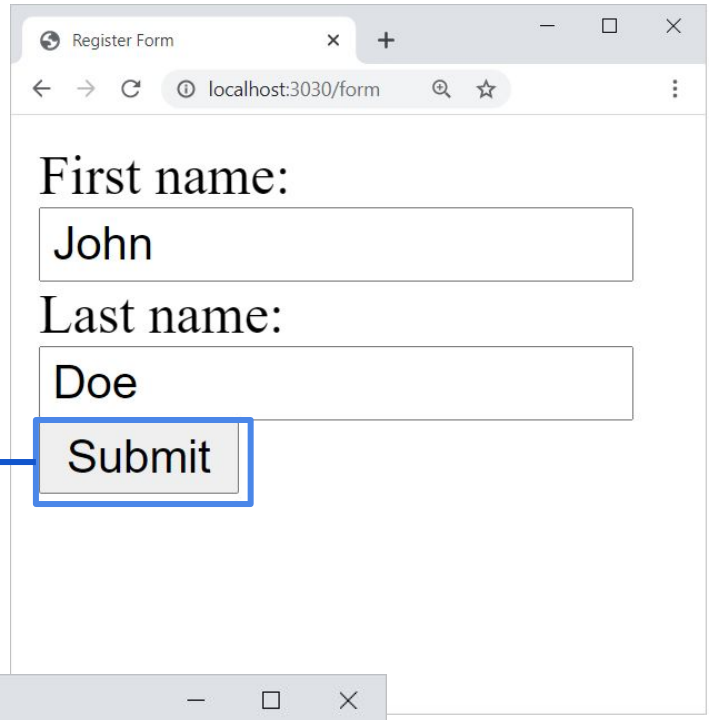
req.body contains the data in a string or JSON object from the client side.

Step 3: Handling POST request


```
[nodemon] 2.0.15
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node app.js`
Server listening at Port 3030
Send a form
POST
{ fname: 'John' lname: 'Doe' },
```

HTTP GET
localhost:3030/form

Routing to myForm.html



HTTP POST
localhost:3030/form-post

Form Submission

Show Response

Mahidol University

Wisdom of the Land

Mahidol University

Cookies and Sessions

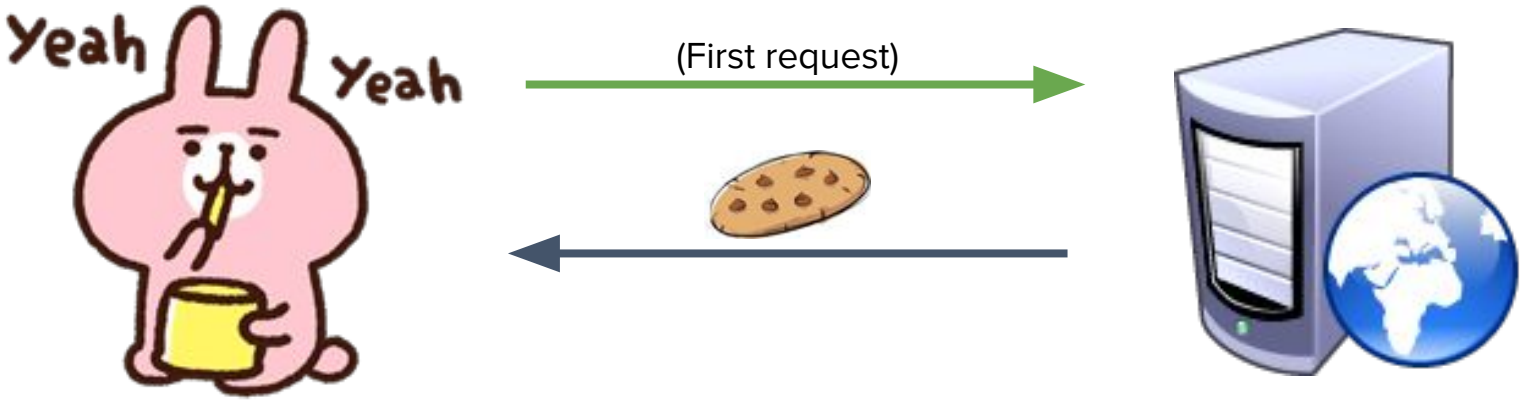
Cookie and Session

- Sometimes, we need to store momentary information to send between pages or to clients

- Two ways to store data
  - **Cookies** (Store information on client's side)
  - **Session** (Store information server's side)

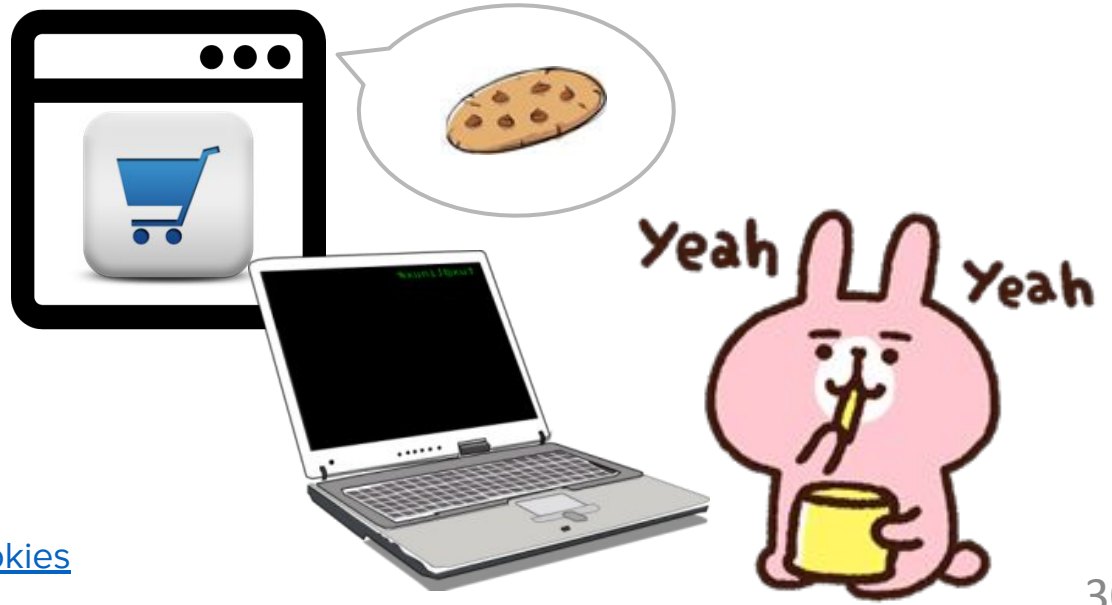
Cookie

- HTTP (HyperText Transfer Protocol) lacks of association between any two HTTP requests, i.e., No method that the client can use to identify itself and the server cannot distinguish between clients.
- Use **Cookie** to keep some data used to identify clients and keep some information specific to each client.



Cookies (cont.)

- Cookie: a small piece of data that a server sends to the user's web browser. The browser may store it and send it back with the next request to the same server.
- Typically tell that two requests came from the same browser
  - Session management
  - Personalization
  - Tracking



Ref: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies

Cookies (cont.)

- Cookies are an extension of the HTTP protocol.
- In Node.js, we can **send** a `Set-Cookie` header with the response
- The cookie is usually stored by the browser. When the cookie is sent with requests made to the same server inside a `Cookie` HTTP header, we can **retrieve** this cookie
- If we use this concept to allow a unique identifier to be included in each request (in a **Cookie header**), you can begin to **uniquely identify clients** and associate their requests together.

Ref: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies

Getting Started with Cookie

- Initialize Express
- To use Cookies in Node.js, the package called **cookie-parser** is required:

```
npm install cookie-parser --save
```

- **cookie-parser** is a middleware which parses cookies attached to the client request object.

- Import "**cookie-parser**" and use it

```
const cp = require("cookie-parser");
router.use(cp());
```

package.json

```
...
"dependencies": {
  "cookie-parser": "^1.4.5",
  "express": "^4.17.1",
  "nodemon": "^2.0.7"
}
```

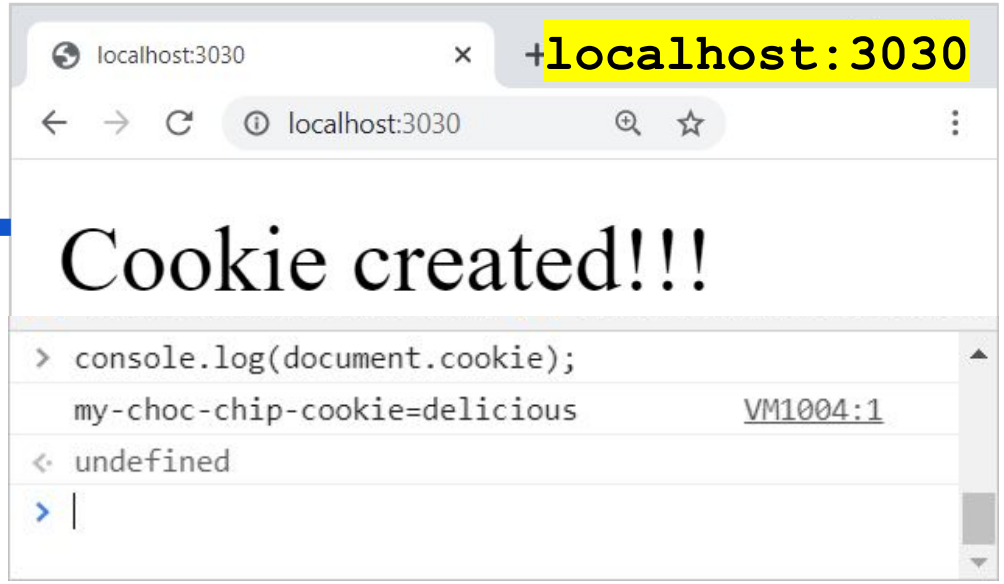
Creating a Cookie

app.js

```
/* Cookies */
const express = require("express");
const app = express();
const router = express.Router();
app.use("/", router); // Register the router

/* Import cookie-parser */
const cp = require("cookie-parser");
router.use(cp()); // Register cookie-parser

router.get("/", function (req, res) {
    /* Create a cookie with the given name and value */
    console.log("Cookies created");
    res.cookie("my-choc-chip-cookie", "delicious");
    res.send("Cookie created!!!");
});

/* Server listening */
app.listen(3030,, function () {
    console.log("Server listening at Port 3030");
});
```

res.cookie create a cookie with name and value

```
[nodemon] starting `node app.js`
Server listening at Port 3030
Cookies created!!
_
```



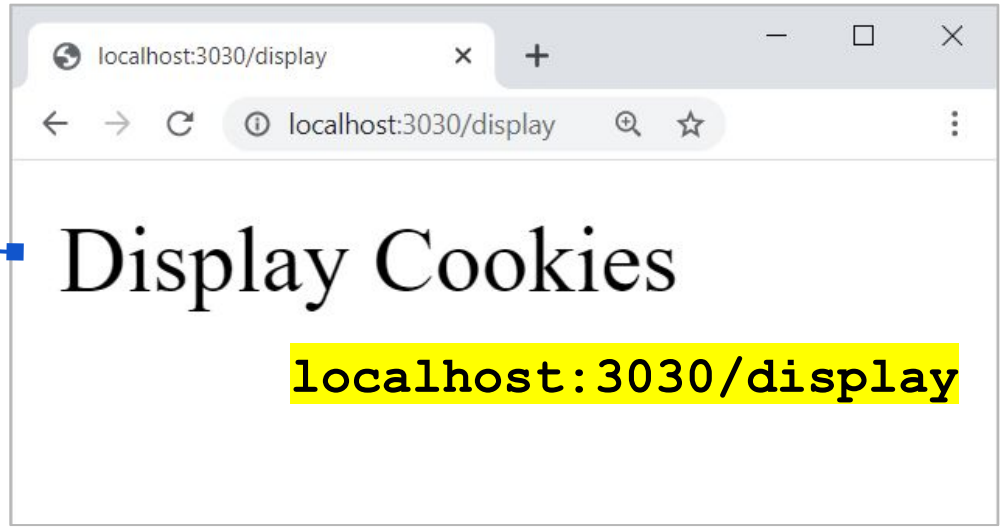
Ref: https://www.tutorialspoint.com/expressjs/expressjs_cookies.htm

Retrieve a Cookie

app.js (cont)

```
...
router.get("/display", function (req, res) {
    /* Display the cookie */
    console.log("Cookies: ", req.cookies);
    res.send("Display Cookies");
});
...
```

```
[nodemon] starting `node app.js`
Server listening at Port 3030
Cookies created!!
Cookies:  { 'my-choc-chip-cookie':
'delicious' }
_
```



`req.cookies` Get the value of all created cookies

`req.cookies["name"]` Get the cookie with the specific **name**, e.g.

```
console.log(req.cookies["my-choc-chip-cookie"]);
// The output is "delicious"
```

More about Cookies

- Set the expiration date or time to the cookie

```
// Expires after 360000 ms from the time it is set.
res.cookie(name, "value", {expire: 360000 + Date.now()});
```

- Set the length of time (age) of the cookie

```
// Expires after 360000 ms after the time is set
res.cookie(name, "value", {maxAge: 360000});
```

- Delete the cookie

```
// Delete the cookie with the specific name e.g. "username"
res.clearCookie("username");
```

Cookies

- Cookies (and URL parameters) can be used to transport data between the client and the server
- However, they are readable and on the client side.

What's about server side?

Session

- Use `session` to hold information about one single user, and are available to all pages in one web application.
- Session can be used to store a unique ID in the cookie and remaining user information on the server.

session.id = 1

session.id = 2

Getting Started with Session

- To use Sessions in Node.js, the package called **`express-session`** is required:

```
npm install --save express-session
```

Note: **`express-session`** works together with **`cookie-parser`**

- **`express-session`** is a middleware to manage sessions and store them in the session storage (by default **`MemoryStore`**)

Note: **`MemoryStore`**, is *purposely* not designed for a production environment and is created for debugging and developing. For production, use some other compatible session stores instead

Ref: https://github.com/expressjs/session

package.json

```
...
"dependencies": {
  "cookie-parser": "^1.4.5",
  "express": "^4.17.1",
  "express-session": "^1.17.1",
  "nodemon": "^2.0.7"
}
```

Use express-session

app.js

```
const express = require('express');
const app = express();
const path = require('path');
const router = express.Router();
app.use('/', router);

const cookieParser = require('cookie-parser');
router.use(cookieParser());

/* Session */
const session = require('express-session');
router.use(session({
    secret: "PutUrSecret",
    resave: false,
    saveUninitialized: true,
}));

// ...to be continued next slide ...
```

There are a lot of options used for session, for example,

- `secret`: your secret string to sign the session ID
- `resave`: forced the session to be saved to the store even though it is not modified
- `saveUninitialized`: Forces a session that is "uninitialized" to be saved to the store.

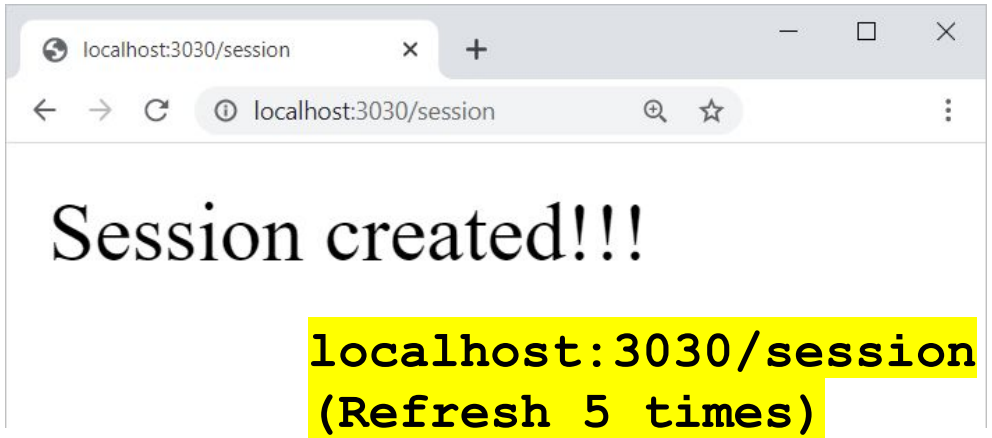
Ref: https://github.com/expressjs/session

Create/Get Session

app.js

```
/* continue from the previous slide */

router.get("/session", function (req, res) {
    /* Set Session */
    // Check "numviews" exists
    if(req.session.numviews){
        req.session.numviews++;
        console.log("Visited "
        + req.session.numviews+ " times");
    }
    else {
        req.session.numviews = 1;
        console.log("First time visited");
    }
    res.send("Session created!!!");
});
/* ...Server listening at Port 3030 ... */
```

```
[nodemon] starting `node app.js`
Server listening at Port 3030
First time visited
Visited 2 times
Visited 3 times
Visited 4 times
Visited 5 times
_
```



localhost:3030/session
(Refresh 5 times)

Environmental Variables

Environmental Variable

- Environmental Variables aka. Configuration
- Why we need it?

  If your application need to be run on any computer or cloud, externalizing all environment specific aspects of your app and keep your app encapsulated.
- Creating an `.env` file help you organizing and maintaining your environment variables
- Same source codes can be deployed anywhere
- When we use it?

  Any places in your code that will be changed based on the environment, such as, port, path, status of project.

Ref:https://medium.com/the-node-js-collection/making-your-node-js-work-everywhere-with-environment-variables-2da8cdf6e786

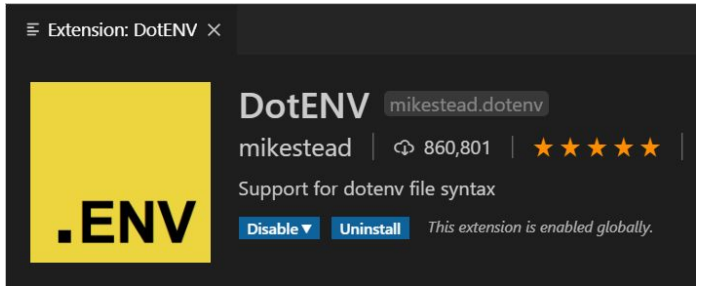
Creating `.env` file

- Create an `.env` file in the root dir
- Add a list of the environmental variables in the file
- For the VSCode, you can install a **DotENV** extension for formatting your `.env` files

```
(root)
> node_modules
|_.env
|_package-lock.json
|_package.json
|_app.js
|_myForm.html
```

.env

```
# Node.js enviromental variables
NODE_ENV=development
PORT=8888
# Set your database/API connection information here
API_KEY=**************************
API_URL=**************************
```



Process the .env file

- To process the .env file, first install the **dotenv** package

```
npm install dotenv --save
```

- **dotenv** is a zero-dependency module that loads the environment variables from the .env file into process.env
- Use the following code for the environmental variables

package.json

```
...
"dependencies": {
  "cookie-parser": "^1.4.5",
  "dotenv": "^8.2.0",
  "express": "^4.17.1",
  "express-session": "^1.17.1",
  "nodemon": "^2.0.7"
}
```

app.js

Refer to **PORT** in .env

```
/* Import and use Environmental Variable */
const dotenv = require("dotenv");
dotenv.config();
/*... More lines hidden here ... */
// Server listening
app.listen(process.env.PORT, function () {
    console.log("Server listening at Port "
    + process.env.PORT);});
```

Mahidol University
Wisdom of the Land

Mahidol University

Node.js + Database

RECAP Your DB Class

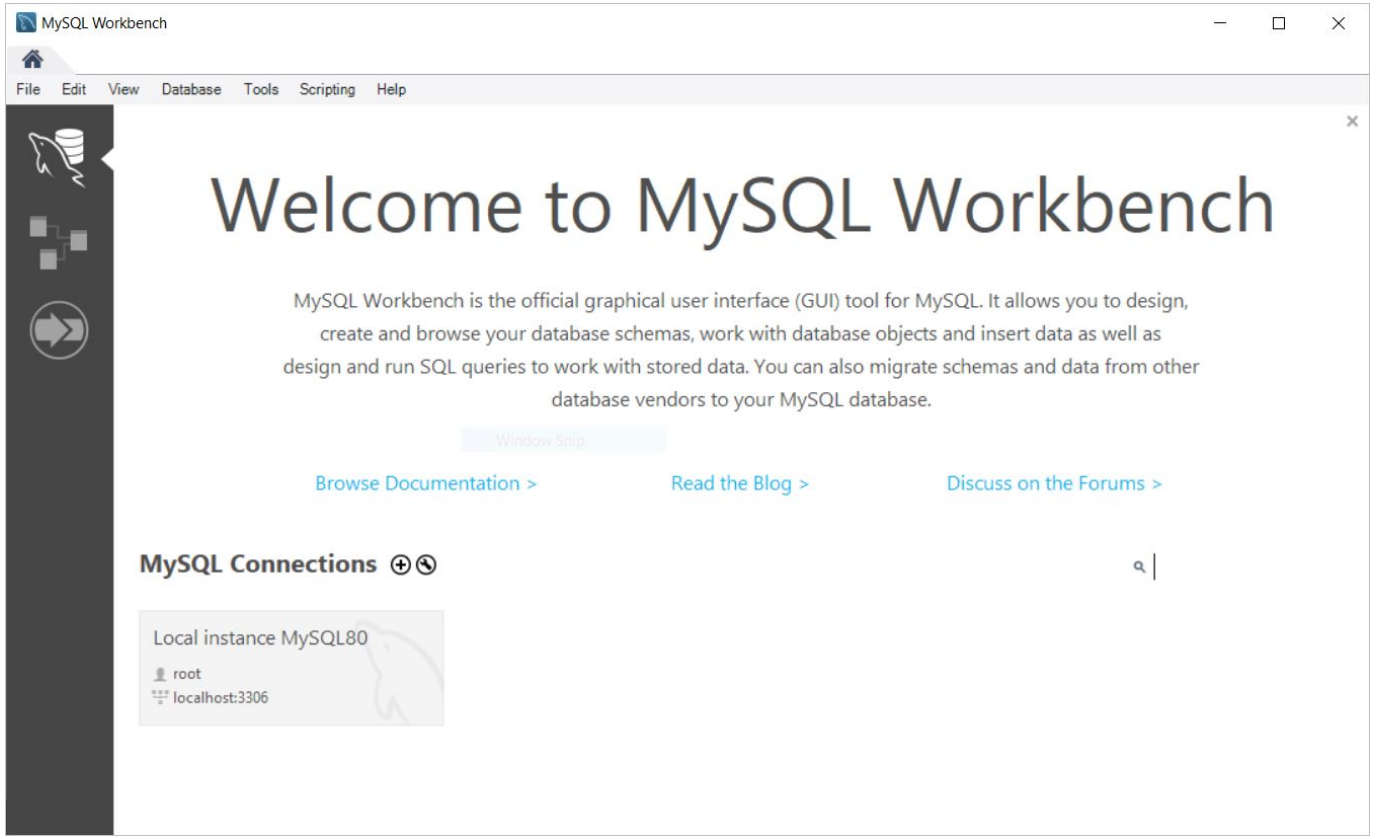
- A **database** is an organized collection of data, which has many types including Relational database, Graph database, XML database, etc.
- A **relational database** is a collection of schemas, tables, queries, reports, views, and other elements.

- A standard language for storing, manipulating and retrieving data in databases and also manage the database in the database management system

- Basic SQL Command
  - **INSERT**
  - **UPDATE**
  - **DELETE**
  - **SELECT**
    - WHERE
    - GROUP BY
    - HAVING
    - ORDER BY

RECALL: MYSQL Server

- Use MySQL Workbench to access your MySQL server for managing your relational database



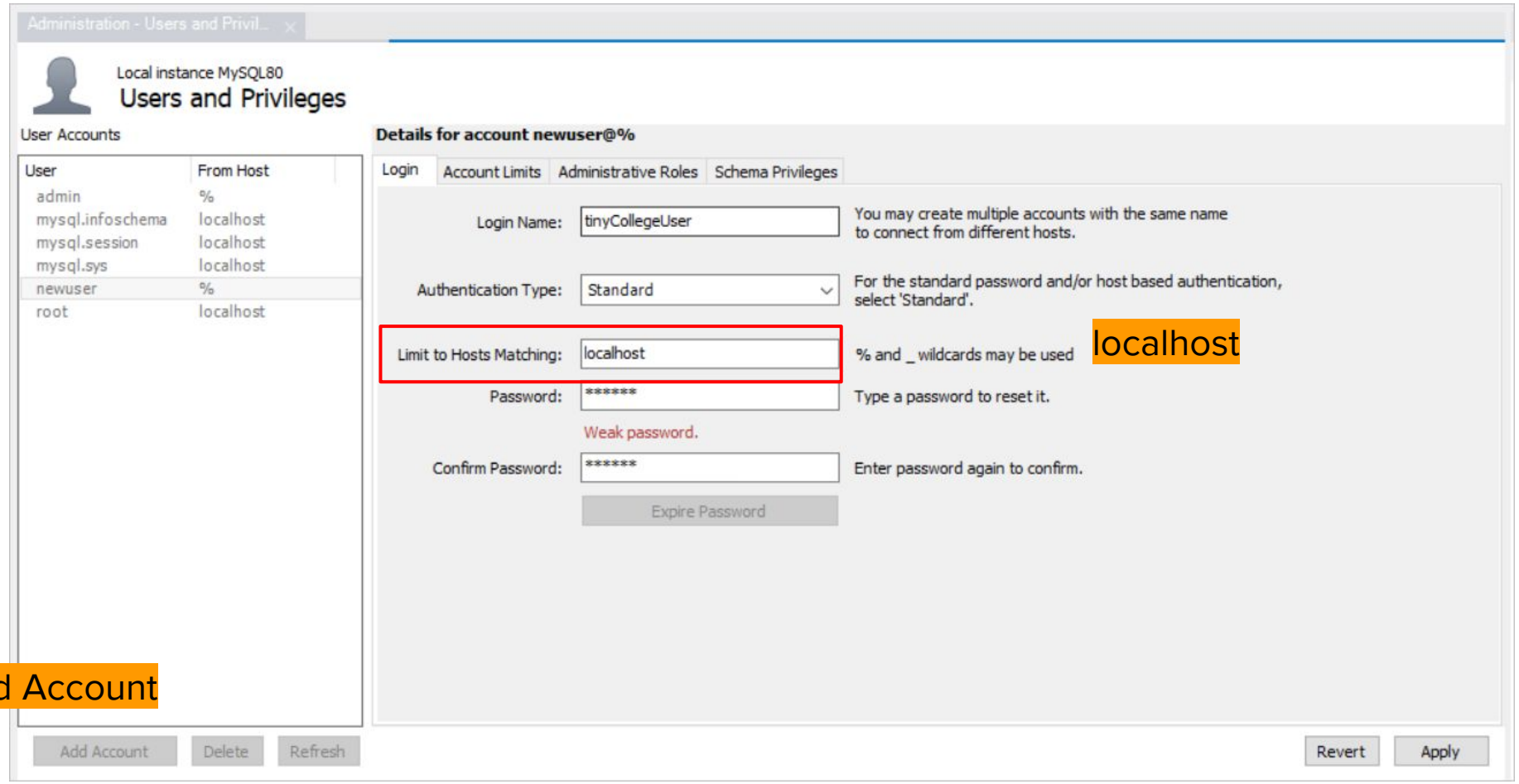
Prepare DB and User

- Create DB/tables/attributes by using MySQL Workbench GUI or running the script (e.g. `tinycollege.sql`)

- Create a user account to be used for connection
  - Username
  - Password
  - Host, i.e. localhost
  - Add privilege i.e., SQL commands, DDL commands

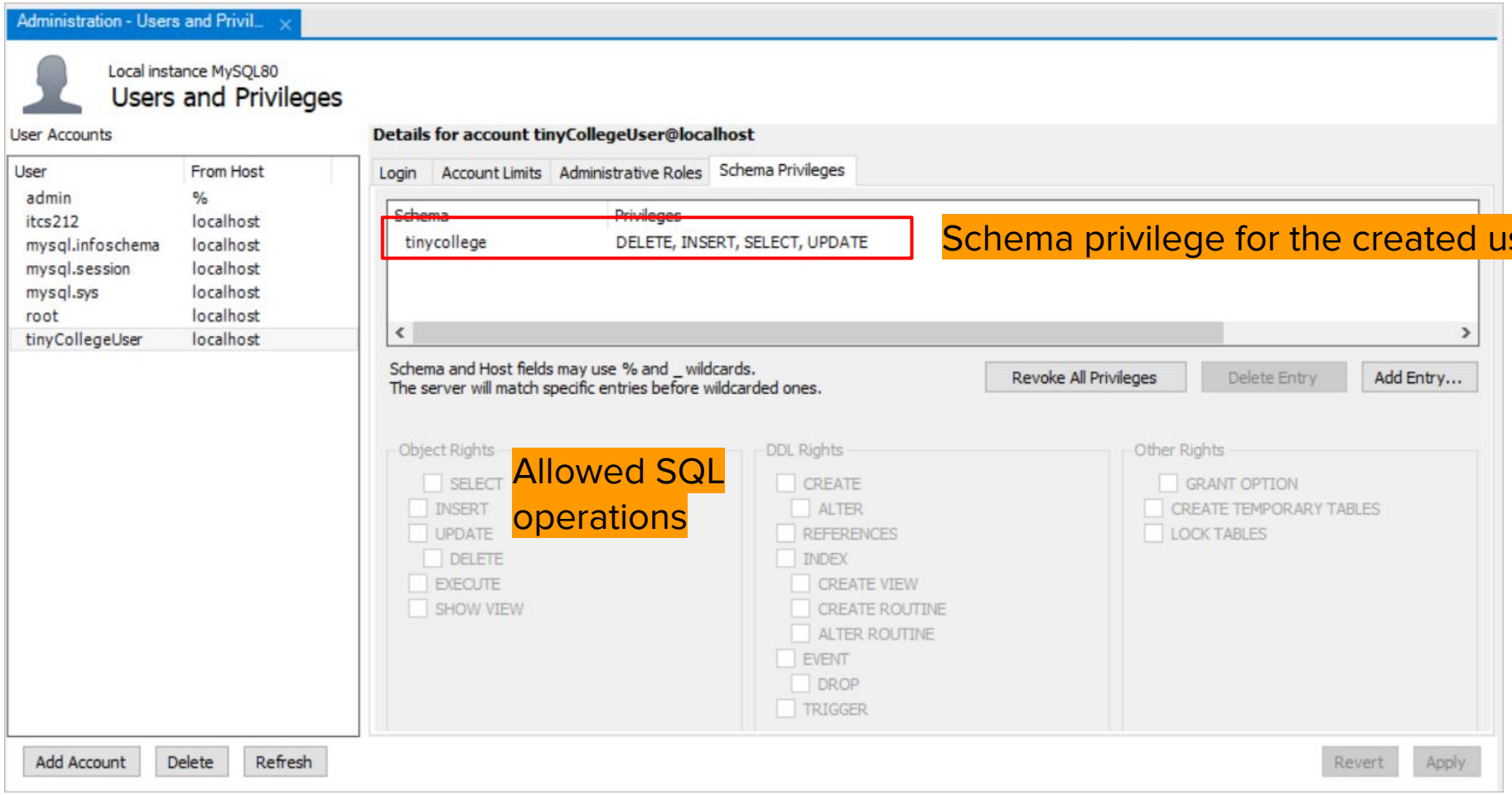
Prepare DB user (cont.)

- In MySQL Workbench, go to “Server” > “Users and Privileges”



Prepare DB user (cont.)

- Set the schema privilege/roles



Getting Started with DB+Node.js

- There are several packages for connecting MySQL to Node.js express e.g. `mysql`, `mysql2`

- In our class, we use the package called **`mysql2`**

```
npm install mysql2 --save
```

- Use this to connect to the database and manipulate data via SQL, e.g., query and show data records, insert a new record, update and delete the data.

Example: DB + Node

- Create a new project, i.e.,

```
npm init
npm install express --save
npm install dotenv --save
npm install mysql2 --save
```

package.json

```
...
"dependencies": {
  "dotenv": "^8.2.0",
  "express": "^4.17.1",
  "mysql2": "^2.2.5",
  "nodemon": "^2.0.7"
}
```

- Create an entry point file, i.e., `app.js`, with a basic routing and server running
- Create **`.env`** file with the variables such as `PORT`, `DB_USER`, `DB_PWD`, `DB_HOST` etc.

Example: DB + Node

app.js

```
/* Import modules here: express, dotenv, router */
/* Config dotenv and router */

/* Connection to MySQL */
const mysql = require('mysql2');
var connection = mysql.createConnection({
    host     : process.env.MYSQL_HOST,
    user     : process.env.MYSQL_USERNAME,
    password : process.env.MYSQL_PASSWORD,
    database : process.env.MYSQL_DATABASE
});

/* Connect to DB */
connection.connect(function(err){
    if(err) throw err;
    console.log("Connected DB: "+process.env.MYSQL_DATABASE);
});

/* Run Server */
app.listen(process.env.PORT, function() {...});
```

.env

```
# Node Env. Var
PORT=8080

# For DB connection string
MYSQL_HOST=localhost
MYSQL_USERNAME=tiny
MYSQL_PASSWORD=************
MYSQL_DATABASE=tinycollege
```

```
[nodemon] starting `node
app.js`
Server listening at Port 8080
Connected DB: tinycollege
_
```

Example: SELECT - JSON

app.js

```
...
/* SELECT */
router.get("/cis_courses", function (req, res) {
    console.log("Retrieved CIS courses in tinycollege...")
    let sql = "SELECT course_code, course_name
               FROM Course WHERE dept_code='CIS'";
    connection.query( sql, function (error, results) {
            if (error) throw error;
            console.log(results.length + " rows returned");
            return res.send(results);
    });
});
```

```
[nodemon] starting `node app.js`
Server listening at Port 8080
Connected DB: tinycollege
Retrieved all courses in
tinycollege...
6 rows returned
```

localhost:3030/courses


```
[{"course_code":"CIS-220","course_name":"Intro.
to Microcomputing"},{"course_code":"CIS-
320","course_name":"Spreadsheet Applications"},
{"course_code":"CIS-370","course_name":"Intro. to
Systems Analysis"},{"course_code":"CIS-
420","course_name":"Database Design and
Implementation"},{"course_code":"QM-
261","course_name":"Intro. to Statistics"},
{"course_code":"QM-
362","course_name":"Statistical Applications"}]
```

Example: INSERT

app.js

```
...
/* SELECT */
router.get("/courses/add212", function (req, res) {
    console.log("Adding CIS-212 ...");
    let sql = "INSERT INTO Course
               VALUES('CIS-212','Web Programming',3,'CIS')";
    connection.query( sql, function (error, results) {
            if (error) throw error;
            console.log(results.affectedRows
                        + " record inserted");
            return res.redirect("/courses");
            /* Go back to show all courses at /courses */
    });
});
```

```
[nodemon] starting `node app.js`
Server listening at Port 8080
Connected DB: tinycollege
Retrieved all courses in
tinycollege...
6 rows returned
Adding CIS-212 ...
1 record inserted
Retrieved all courses in
tinycollege...
7 rows returned
```

[Extra] mysql2

- Learn more about other sql command
  - INSERT https://www.w3schools.com/nodejs/nodejs_mysql_insert.asp
  - UPDATE https://www.w3schools.com/nodejs/nodejs_mysql_update.asp
  - DELETE https://www.w3schools.com/nodejs/nodejs_mysql_delete.asp

- `mysql2` also support Promise API and works with ES7 `async/await`.

See you all next week